PyTorch自动微分

黑马程序员
www.itheima.com
传智教育旗下
高端IT教育品牌

学习目标

Learning Objectives

掌握自动微分模块的使用

自动微分模块

自动微分（Automatic Differentiation, AD） 是 PyTorch 的核心功能之一，它通过 autograd 模块实现，无需手动计算梯度，极大简化了神经网络训练等依赖梯度优化的任务。

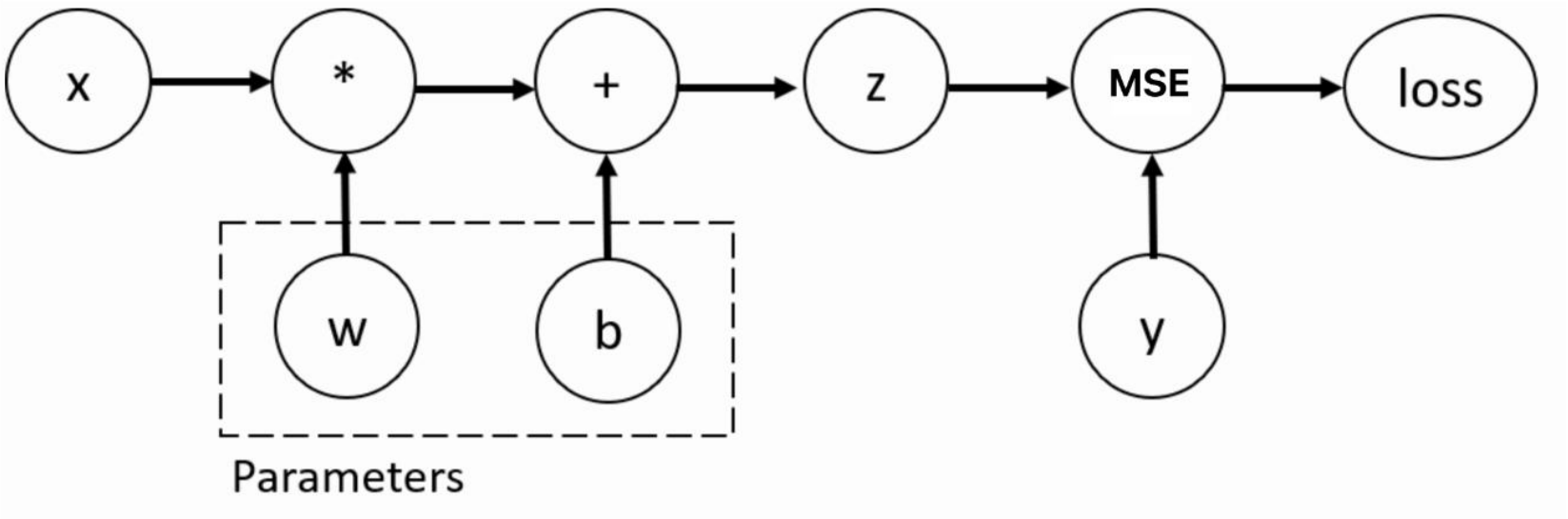
关键点：

1. `torch.autograd`
   PyTorch 的自动微分引擎，在张量计算时自动跟踪操作并构建计算图（动态图），反向传播时自动计算梯度。
2. **如何启用**
   - 通过设置 `requires_grad=True` 标记需要追踪梯度的张量（如模型参数）。
   - 示例：

```
import torch
x = torch.tensor([2.0], requires_grad=True)
y = x ** 2  # 计算图自动记录
y.backward()  # 自动计算梯度
print(x.grad)  # 输出梯度值: tensor([4.0])
```

自动微分模块

训练神经网络时，最常用的算法就是反向传播。在该算法中，参数（模型权重）会根据损失函数关于对应参数的梯度进行调整。PyTorch内置的名为 torch.autograd 的微分模块。它支持任自动梯度计算：



接下来我们使用这个结构进行自动微分模块的介绍。我们使用 backward 方法、grad 属性来实现梯度的计算和访问.

梯度基本计算

- PyTorch不支持向量张量对向量张量的求导，只支持标量张量对向量张量的求导
  - x如果是张量，y必须是标量(一个值)才可以进行求导
- 计算梯度：y.backward()，y是一个标量
- 获取x点的梯度值：x.grad，注意它会累加上一次的梯度值

自动微分(更新w)

```
# 导包
import torch

# TODO 自动微分模块主要用于梯度计算!!!
# 定义权重 requires_grad=True:开启自动微分
w = torch.tensor([10, 20], requires_grad=True, dtype=torch.float)
# 定义损失函数(loss,cost,criterion...)
loss = 2 * w ** 2
# 自动微分求梯度,自动更新梯度
loss.sum().backward()
# 格式化输出
print(f"当前权重: {w.data},固定学习率:{0.01} 更新后梯度: {w.grad},下一个权重: {w.data - 0.01 * w.grad}")
# TODO 手动更新权重
# 公式:  w1 = w0 - learning_rate * grad
w.data = w.data - 0.01 * w.grad
```

输出结果:

```
当前权重: tensor([10., 20.]),固定学习率:0.01 更新后梯度: tensor([40., 80.]),下一个权重: tensor([ 9.6000, 19.2000])

Process finished with exit code 0
```

自动微分(更新w和b)

```
# 准备x训练数据
x = torch.ones(2, 5)
# 准备y训练数据
y = torch.zeros(2, 3)
# 准备w权重矩阵,开启自动微分!!!
w = torch.randn(5, 3, requires_grad=True)
# 准备b偏置矩阵,开启自动微分!!!
b = torch.randn(3, requires_grad=True)
# TODO 最终目的根据上述数据,使用自动微分推导w和b的梯度
# 1.首先获取损失函数
loss_fn = torch.nn.MSELoss()
# 2.然后,计算预测值-> z=wx+b 注意: 这里面的wx是矩阵乘法需要遵循 (n,m)*(m,p)=(n,p)
z = x.matmul(w) + b
# 3.接着,根据损失函数计算损失值
loss = loss_fn(z, y)
# 4.最后,反向传播推导更新梯度
loss.sum().backward()
# TODO 打印更新后w和b梯度
print(f'w.grad: {w.grad}')
print(f'b.grad: {b.grad}')
```

输出结果:

```
tensor([[1., 1., 1., 1., 1.],
        [1., 1., 1., 1., 1.]])
tensor([[0., 0., 0.],
        [0., 0., 0.]])
tensor([[-0.7744, -0.2857, -0.6589],
        [-0.6359, -2.2790, -0.7172],
        [-0.1681,  2.5614, -0.4194],
        [-1.2420, -1.6901, -0.3531],
        [ 0.3059, -0.6458, -0.4324]], requires_grad=True)
tensor([-1.0088, -0.3709, -1.0441], requires_grad=True)
=========================================================
w.grad: tensor([[-2.3490, -1.8068, -2.4168],
        [-2.3490, -1.8068, -2.4168],
        [-2.3490, -1.8068, -2.4168],
        [-2.3490, -1.8068, -2.4168],
        [-2.3490, -1.8068, -2.4168]])
b.grad: tensor([-2.3490, -1.8068, -2.4168])

Process finished with exit code 0
```

总结

本小节主要讲解了 PyTorch 中非常重要的自动微分模块的使用和理解。
我们对需要计算梯度的张量需要设置 requires_grad=True 属性。

黑马程序员
www.itheima.com
传智教育旗下高端IT教育品牌